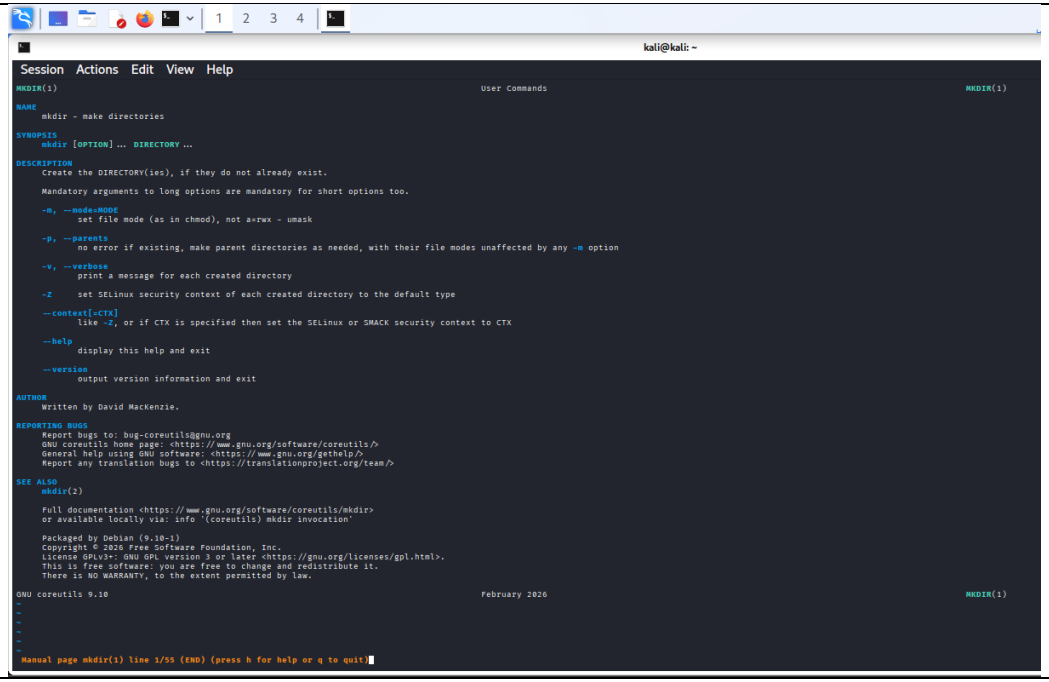
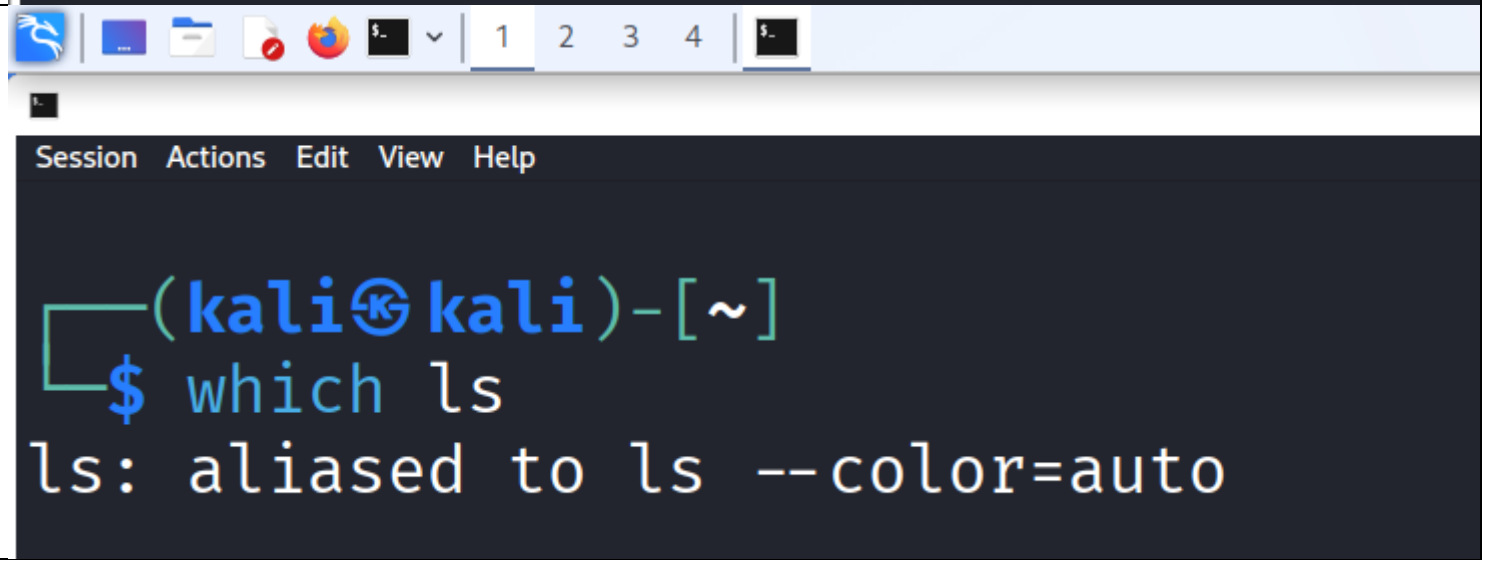
Topic Name:

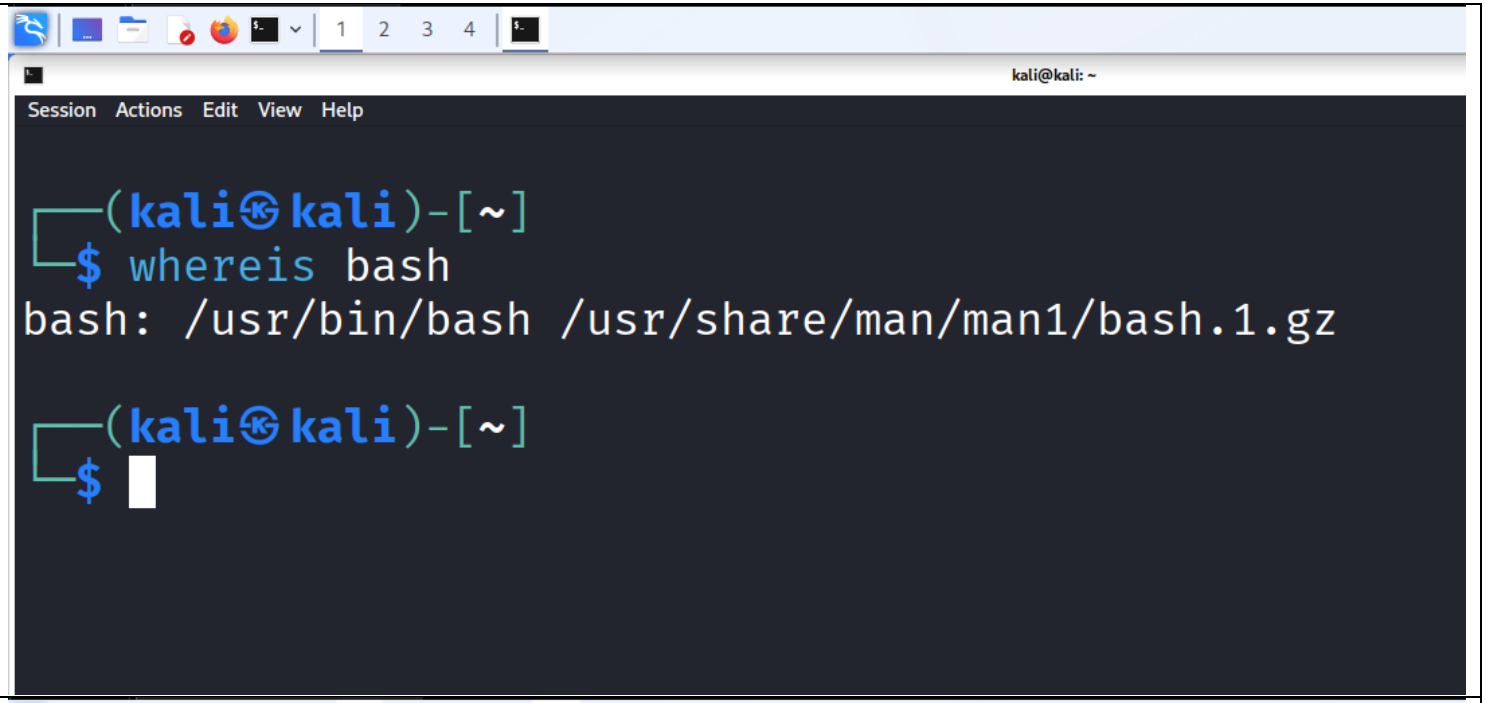
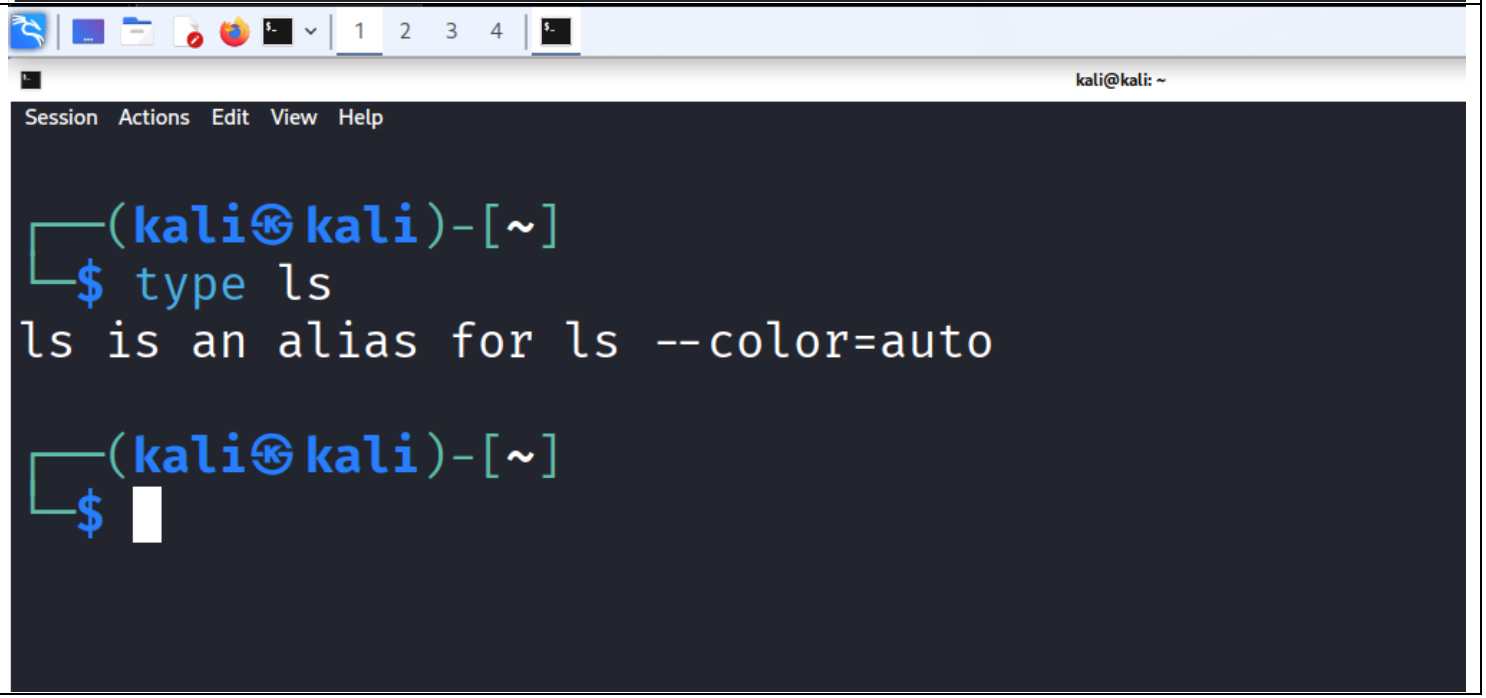
The main aim of this lab session is to provide hands-on experience on

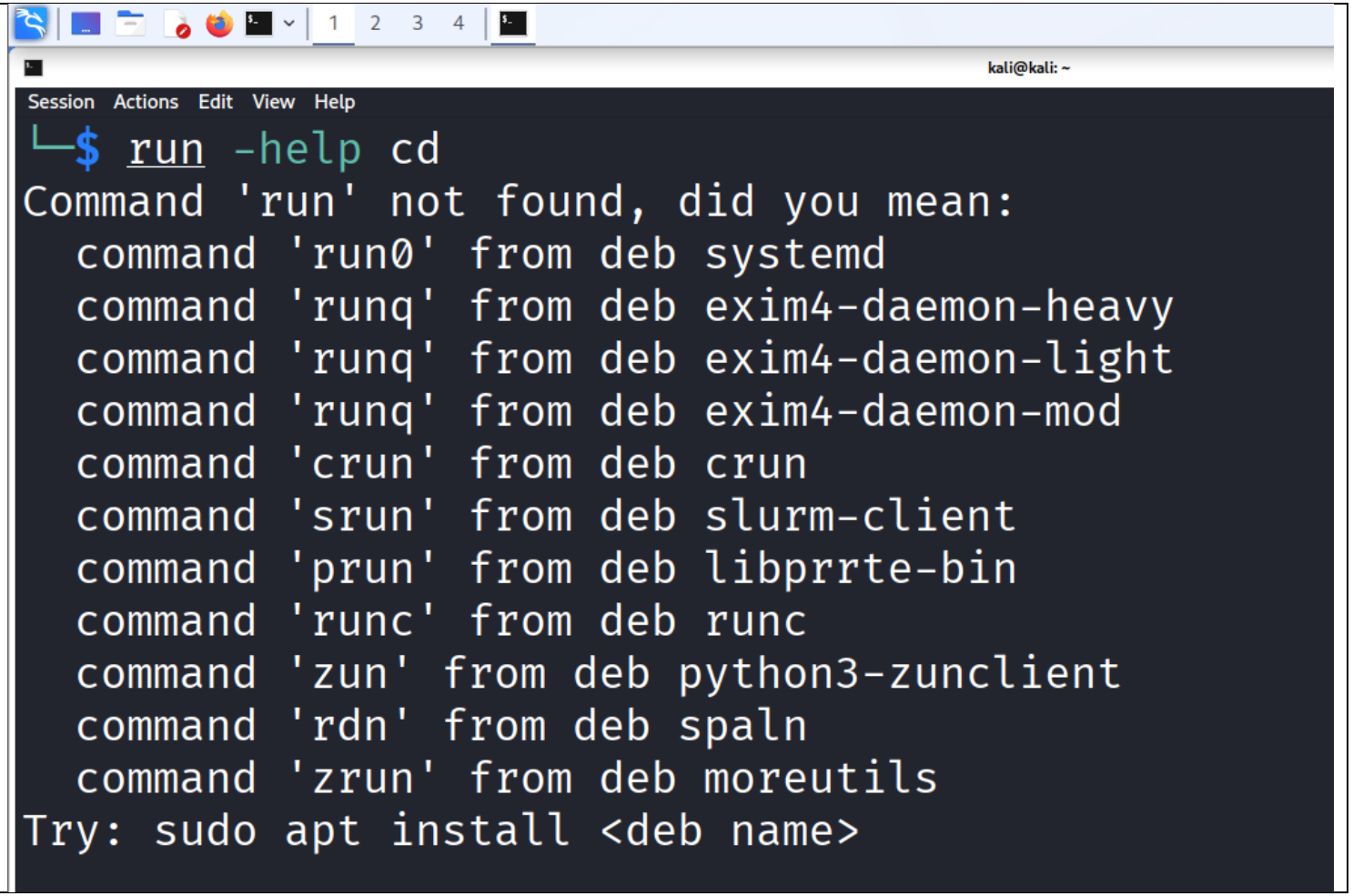
- Getting Help
- Basic Commands
- Navigation
- File System
- Simple shell script
- Globbing
- Extended Globbing
- Filters -I (head,tail,wc)

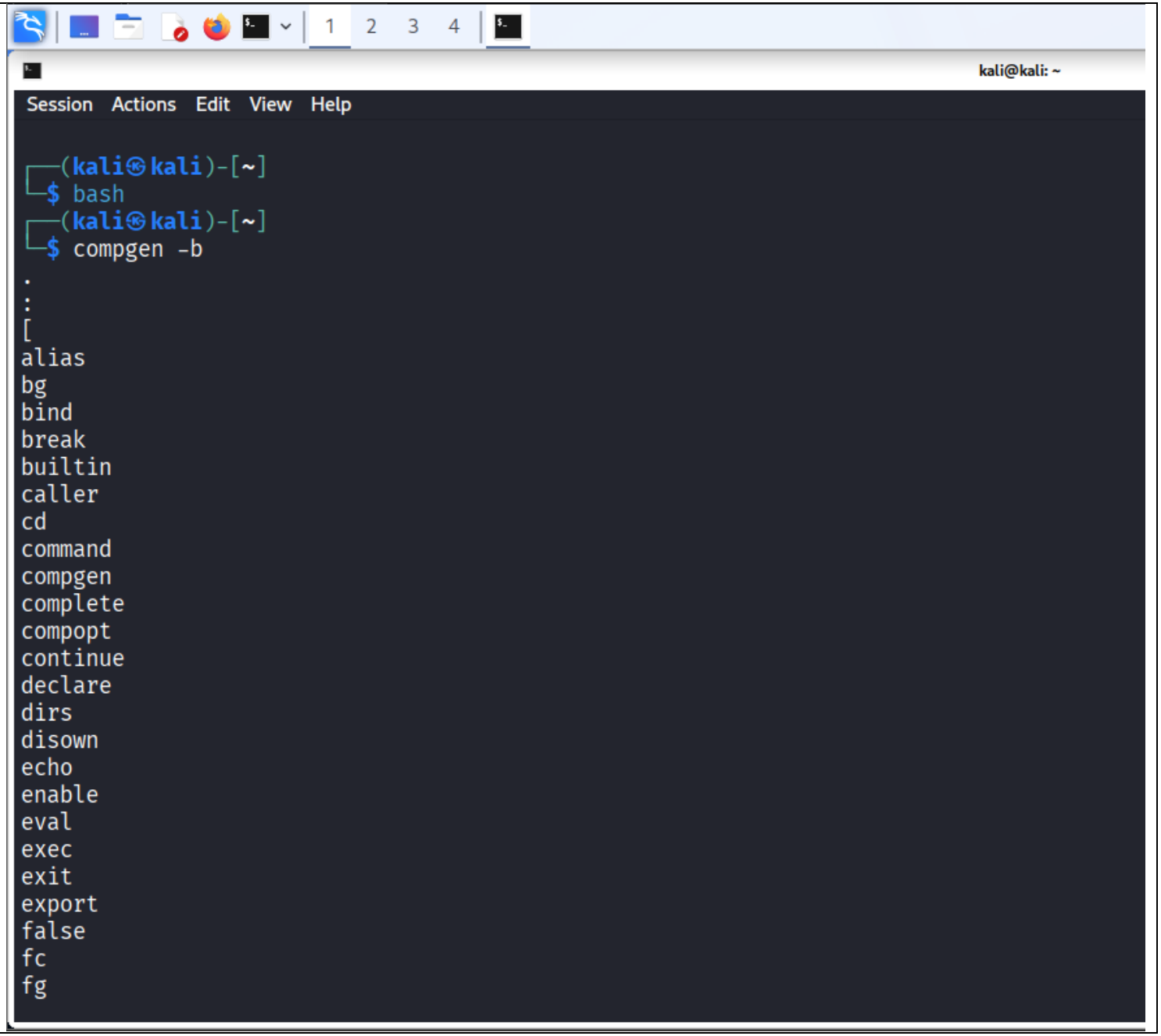
1. Getting Help

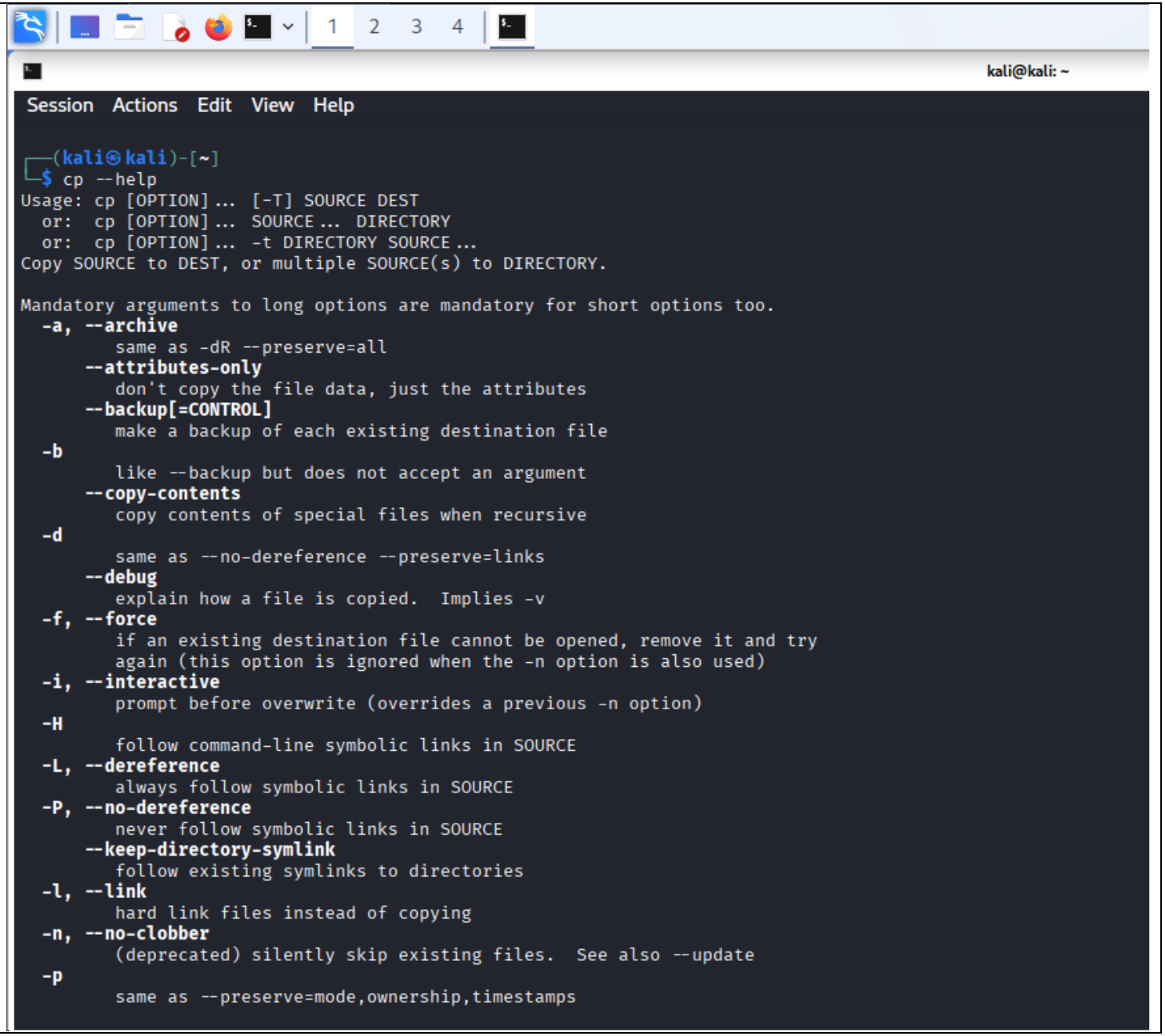
Task	Command Name	Syntax	Example	Screenshots
To get manual page for the known command	man	man command	man mkdir	

To get manual page for the unknown command	man	Man command	man mklr	
To know the source file binary	which	which command	which ls	

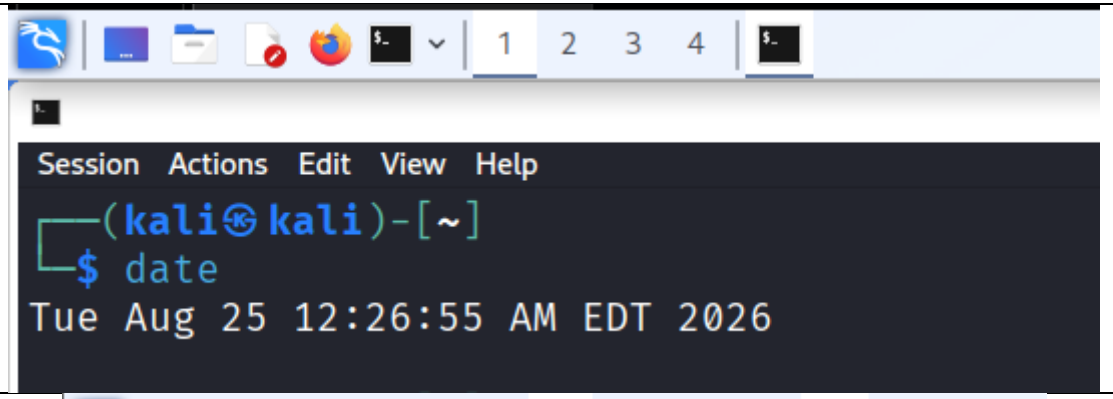
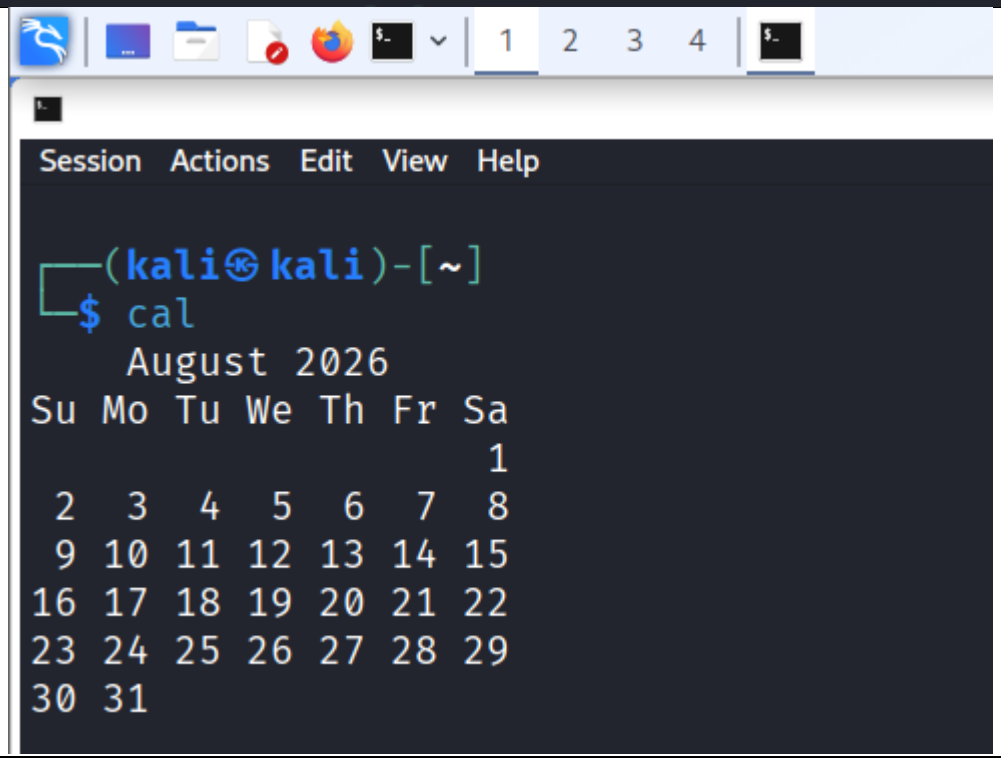
To know the path of the command	whereis	whereis command	whereis bash	
To know the command is external or internal	Type	type command	type ls	

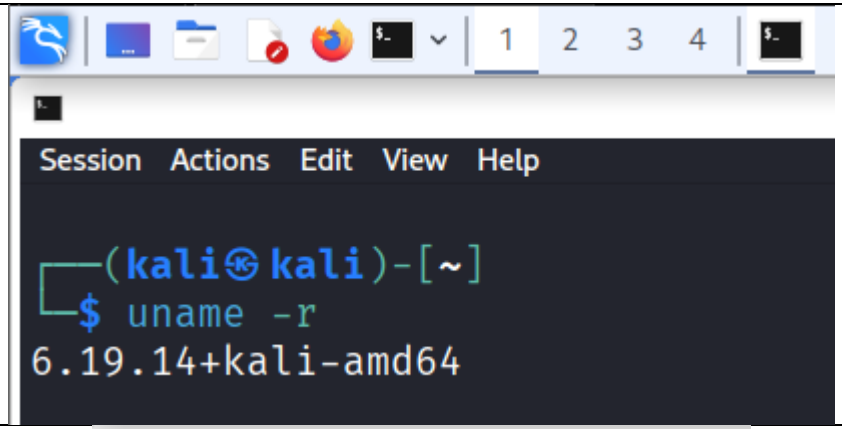
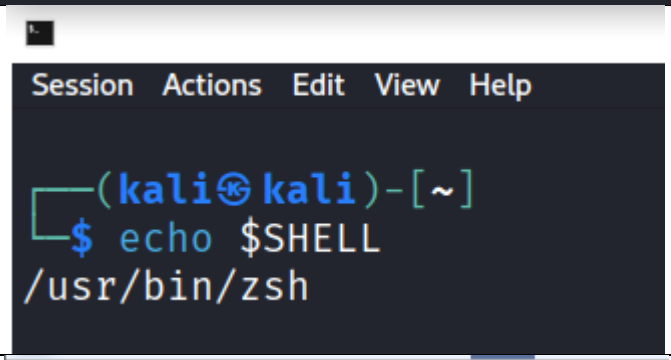
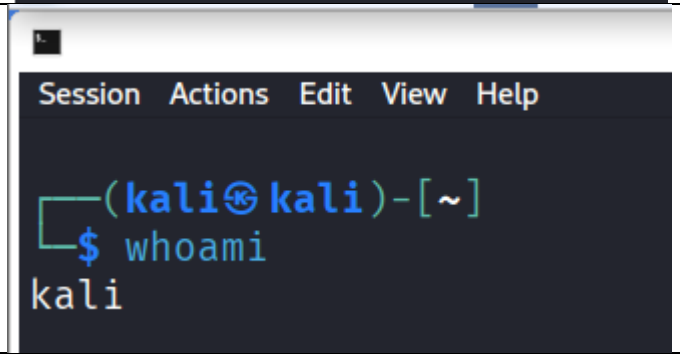
To get help for the internal command	help	run help	run -help cd	

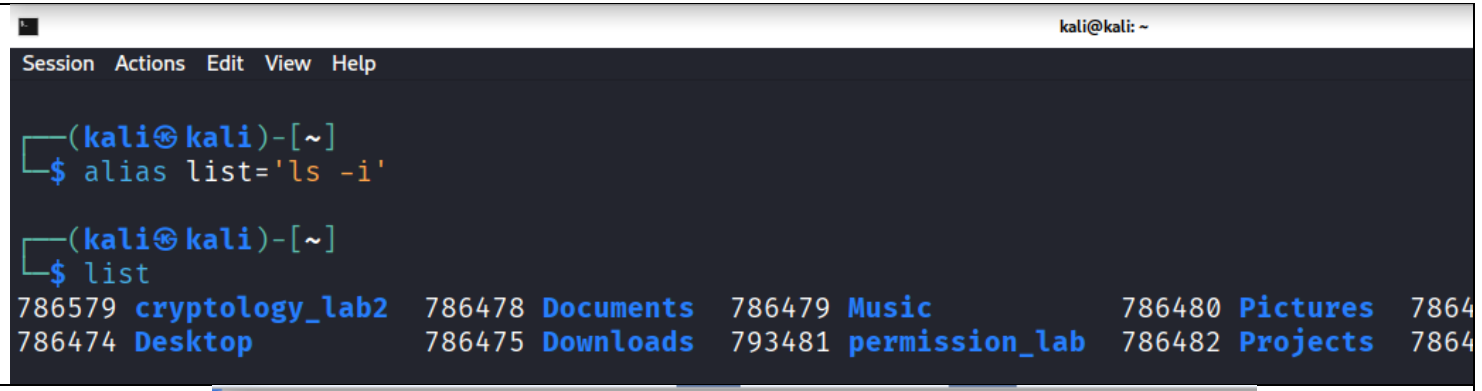
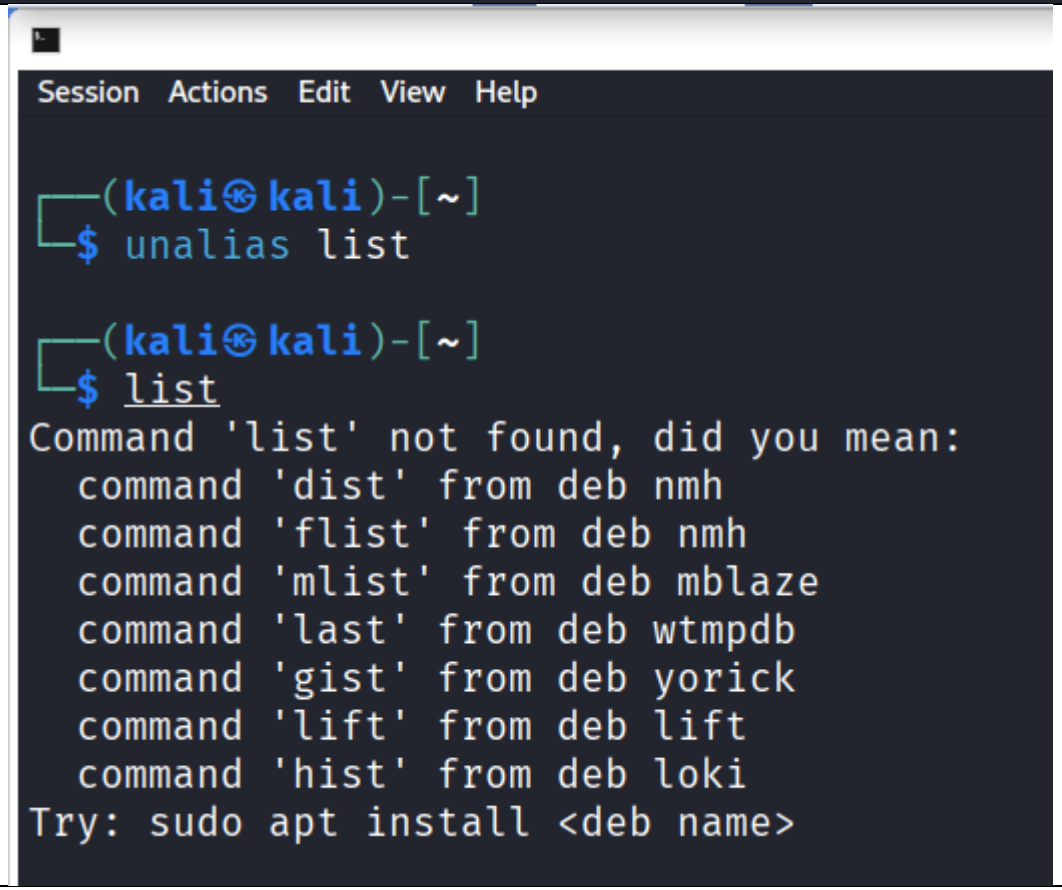
To list out bash commands	compgen	bash compgen -b	bash compgen -b	

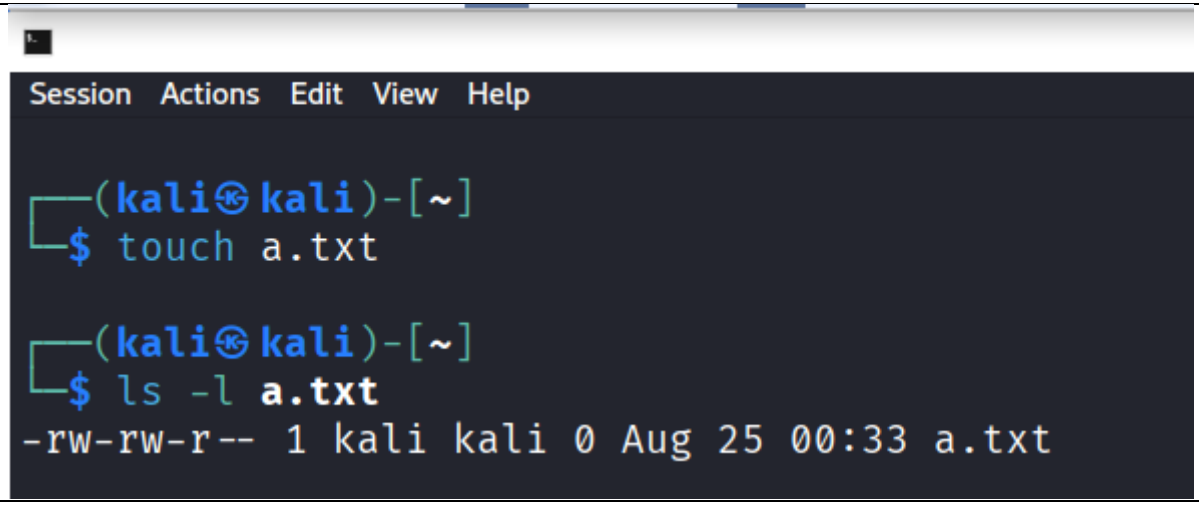
To know the usage of the command	--help	Command --help	cp --help	

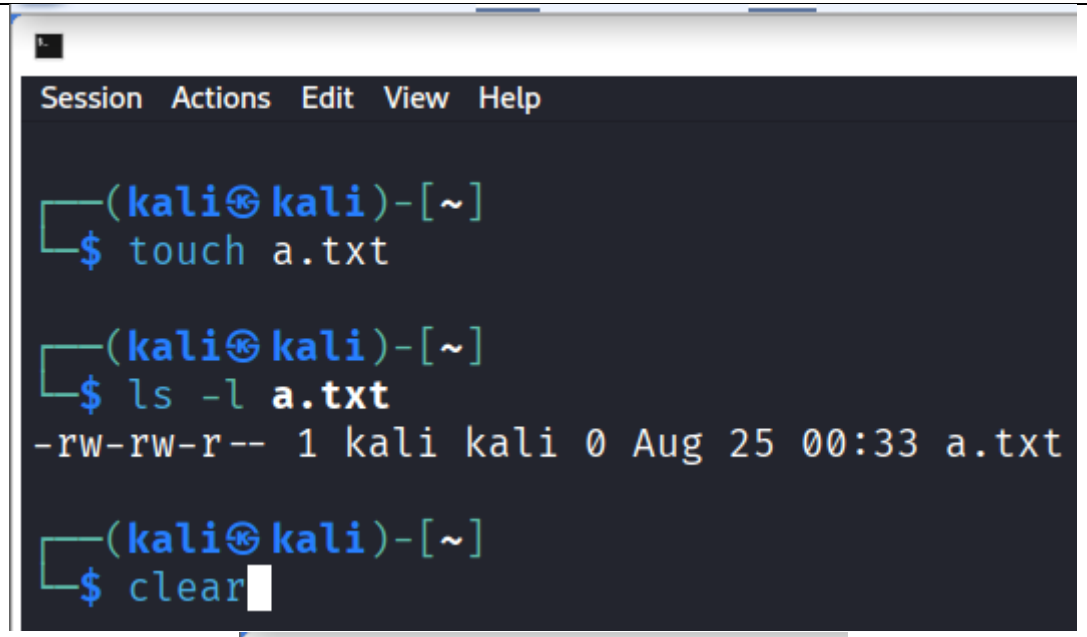
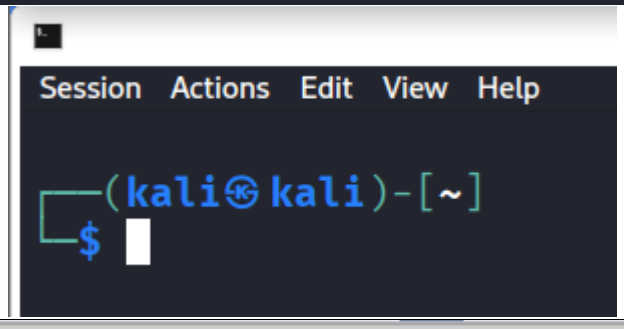
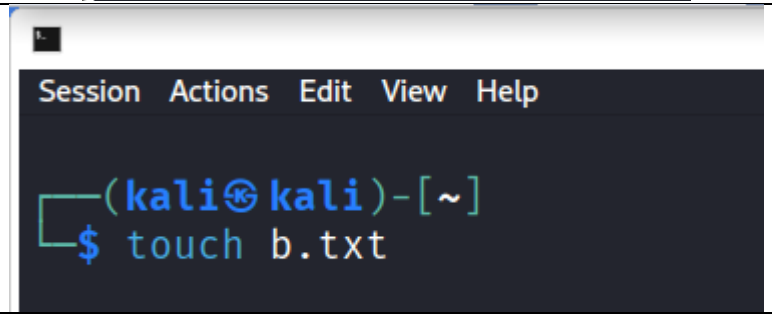
2. Basic Commands

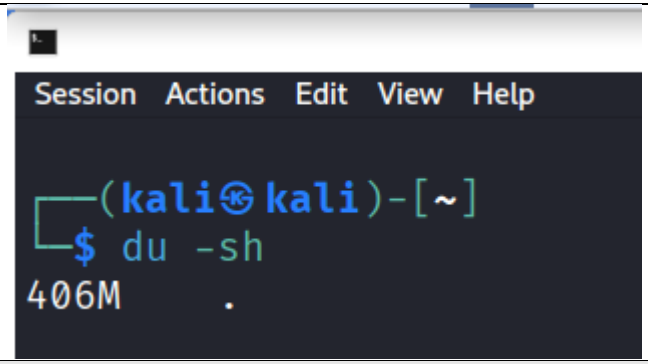
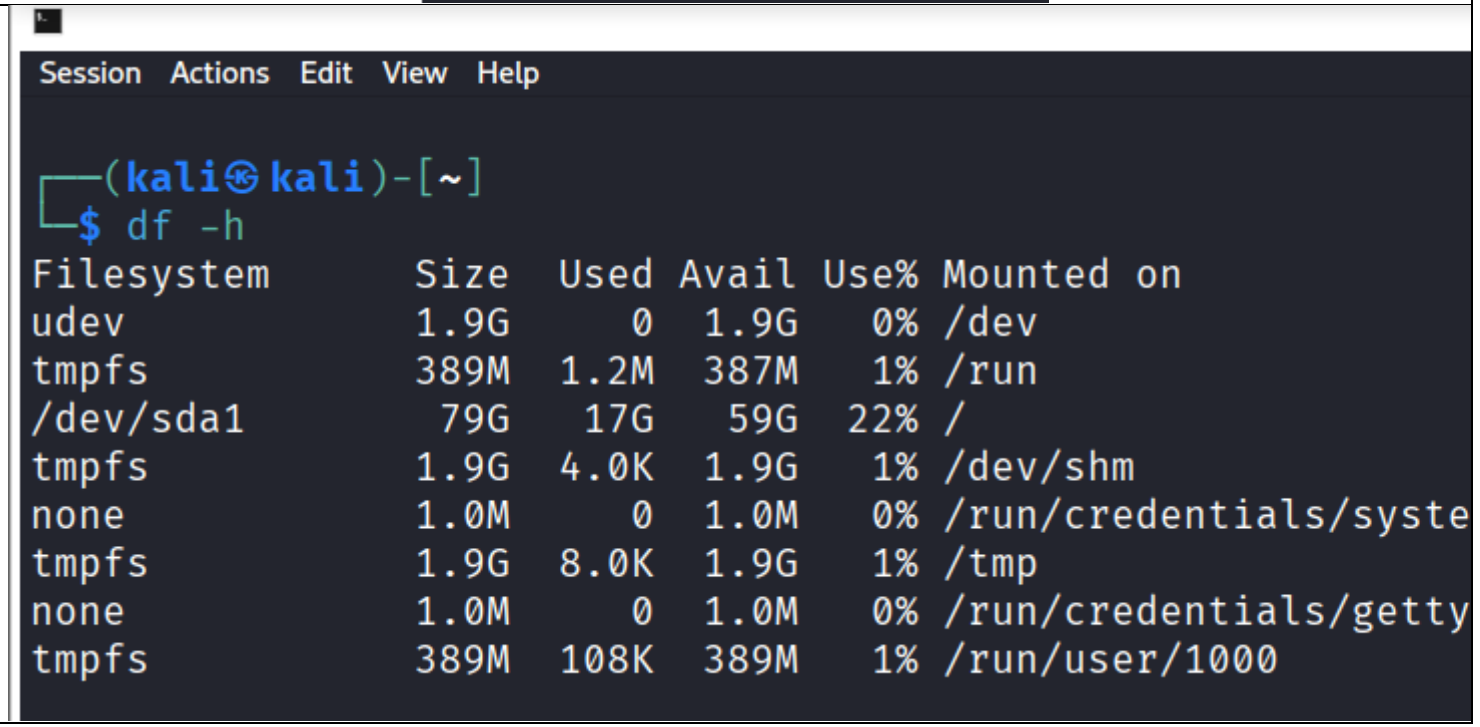
Task	Command Name	Syntax	Example	Screenshots
To know today's date	date	date	date	
To print calendar	cal	cal	cal	

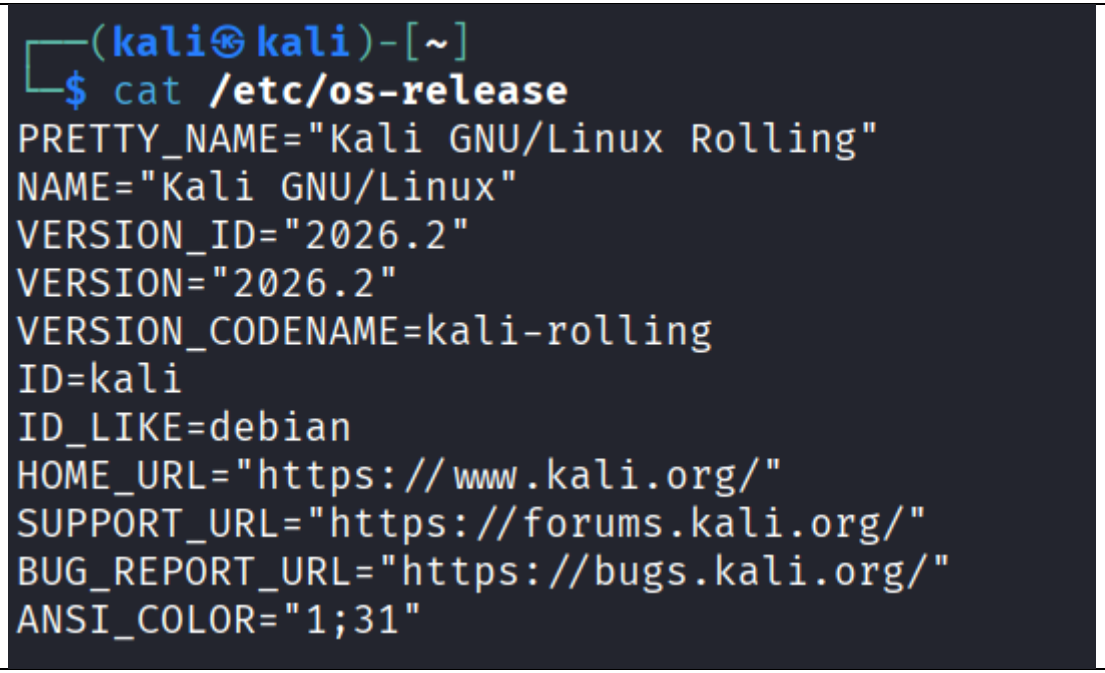
To print kernel version	uname	uname -r	uname -r	
To print default shell	echo	echo $SHELL	echo $SHELL	
To print currently logged in user	whoami	whoami	whoami	

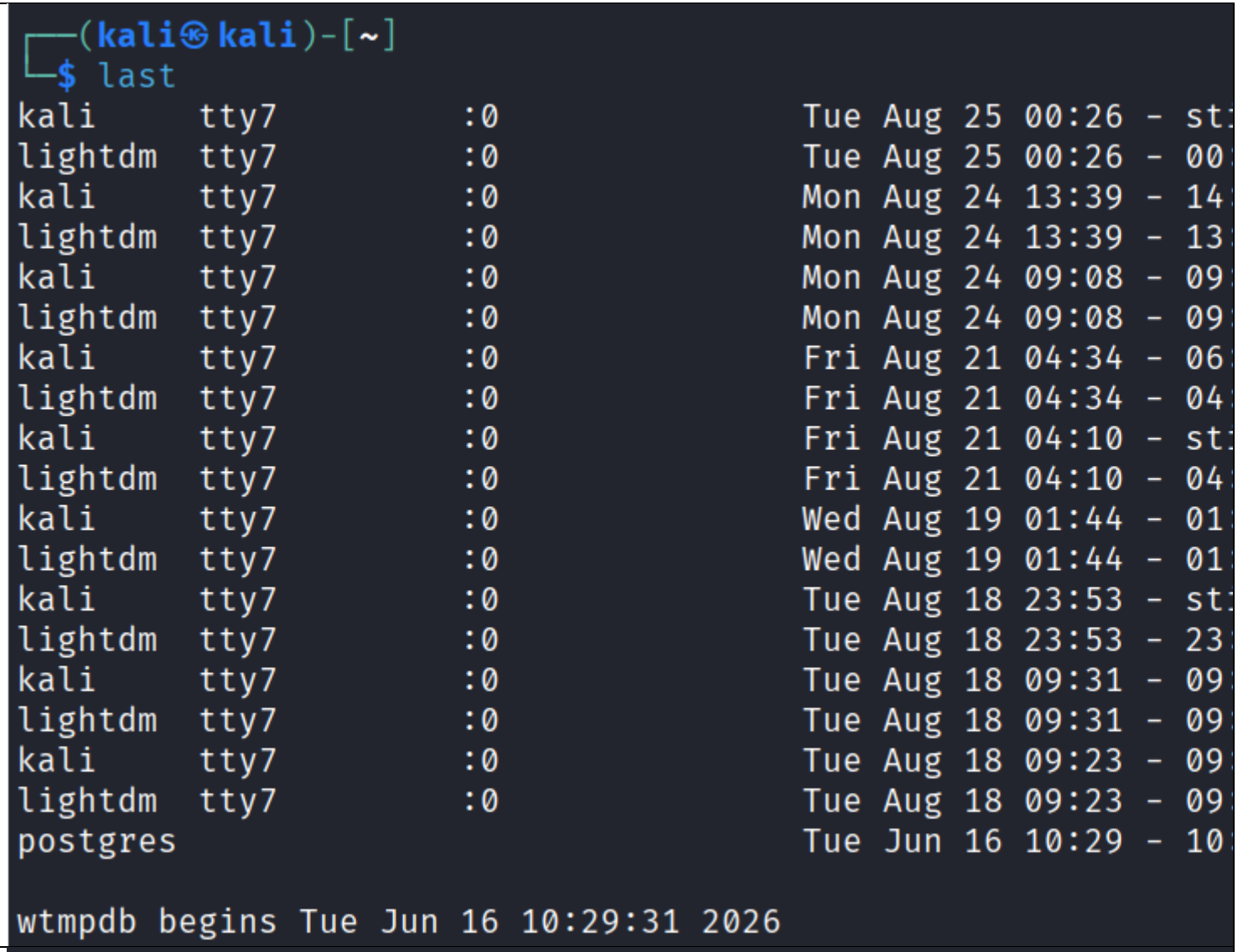
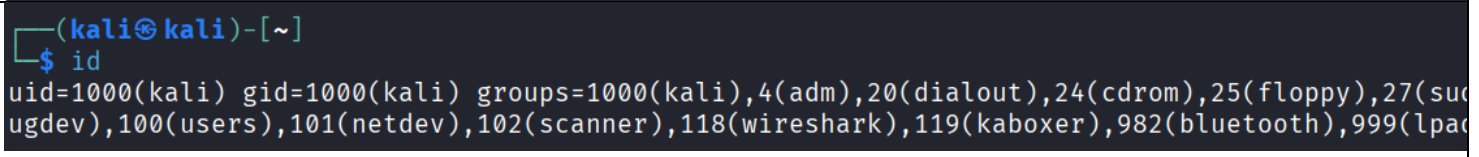
To create shortcut for command	alias	alias name='command'	alias list ='ls -i'	
To delete shortcut	unalias	unalias name	unalias list	

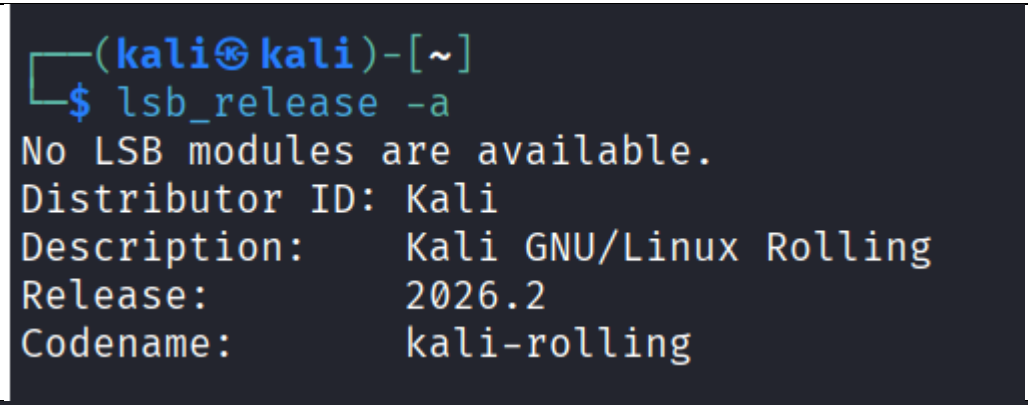
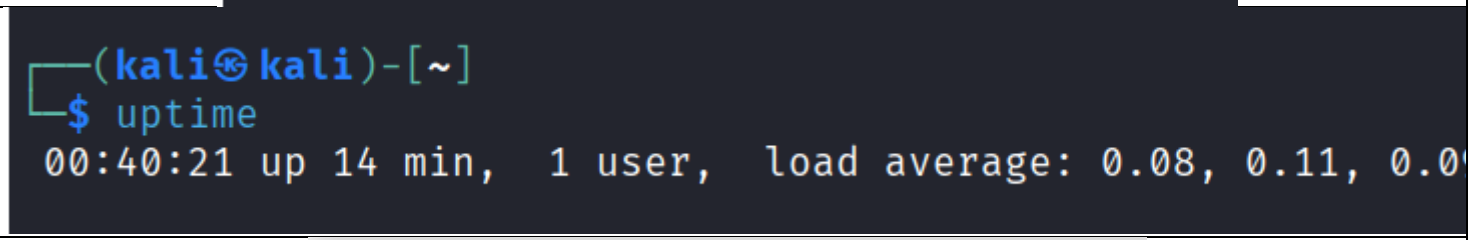
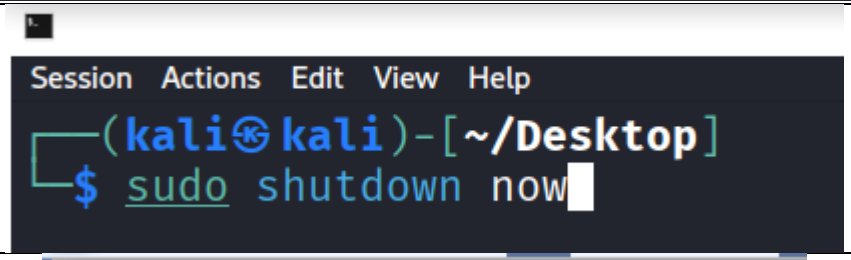
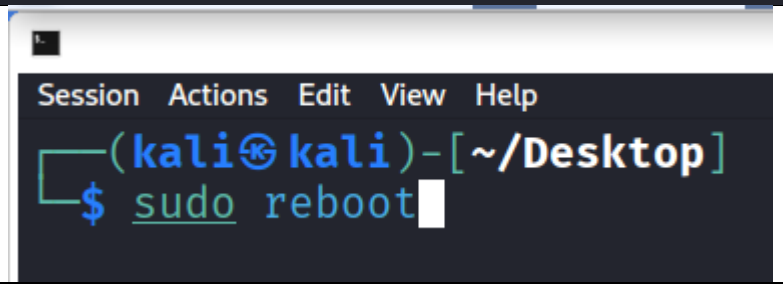
To change the timestamp of the file	touch	touch file	touch a.txt ls -l a.txt	

To clear the screen	clear	clear	clear	 
To create empty files	touch	touch filename	touch b.txt	

To know disk usage	du	du -sh	du -sh	
To know free space in the system	df	df -h	df -h	

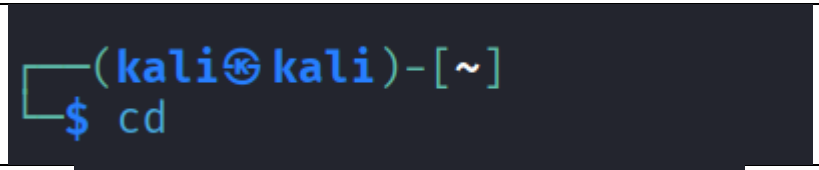
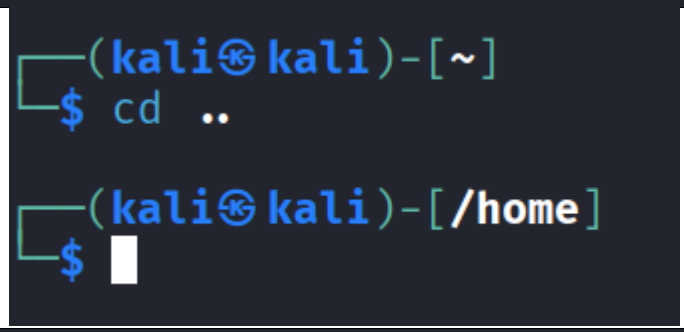
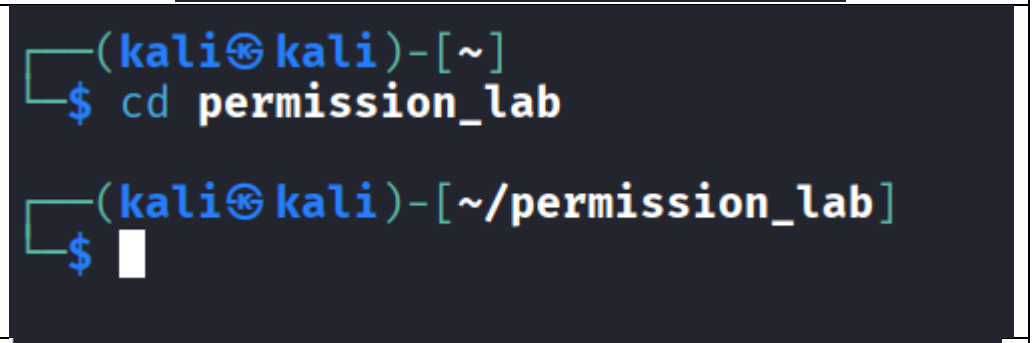
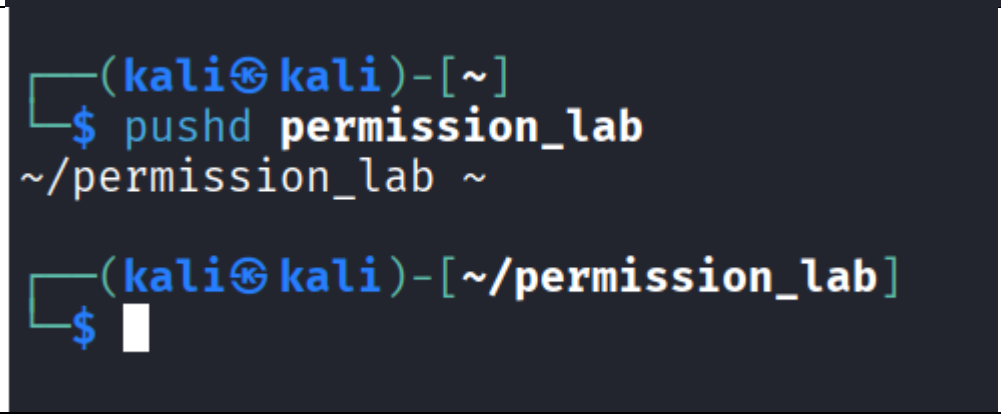
To know about the Linux release	cat	cat/etc/os-release	cat/etc/os-release			

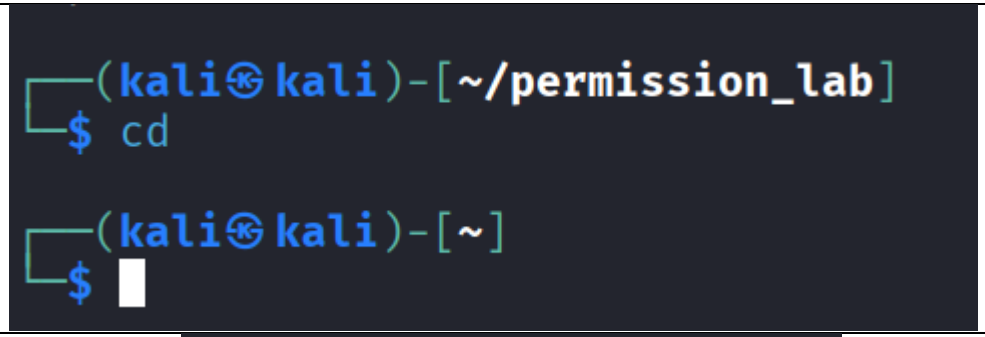
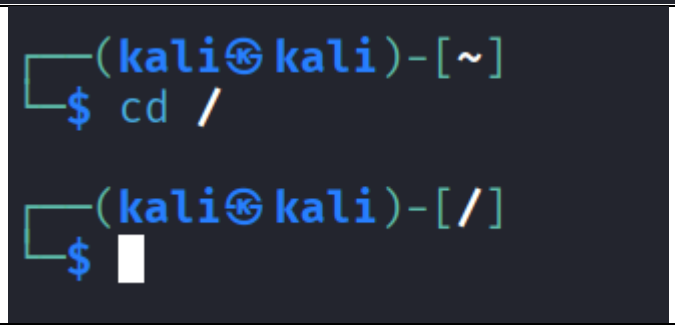
To show login history	last	last	last	
To check the current user privileges	id	id	id	

To find about the Linux release	lsb_release	lsb_release -a	lsb_release -a	
To check system uptime	uptime	uptime	uptime	
To shutdown	shutdown	Sudo shutdown now	Sudo shutdown now	
To restart	reboot	Sudo reboot	Sudo reboot	

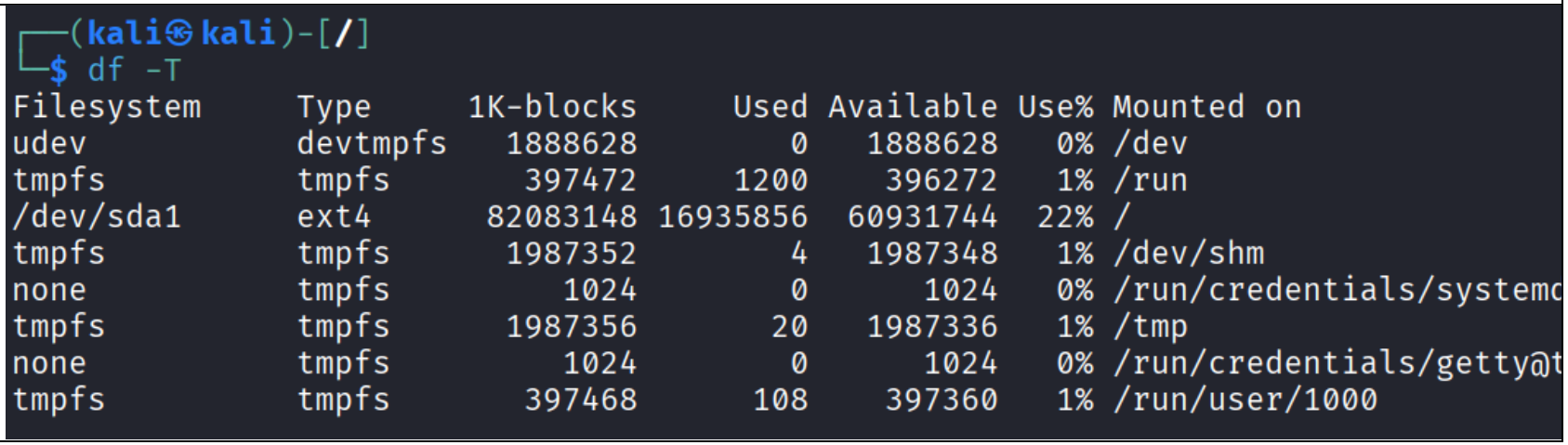
3. Navigation

Task	Command	Syntax	Screenshots

To navigate home directory	cd	cd	
To navigate to the parent directory	cd ..	cd ..	
To navigate to the child directory	cd	cd foldername	
Alternate command to cd	pushd	push directory	

To go back to the previous directory	cd -	cd -	
To go to the root directory	cd /	cd /	

4. **File System**

Task	Syntax	Command	Screenshot
How to identify the file system	**df -T**	**df -T**	

```
(kali㉿kali)-[/]
$ df -T
Filesystem     Type     1K-blocks     Used Available Use% Mounted on
udev           devtmpfs   1888628        0   1888628   0% /dev
tmpfs          tmpfs       397472     1200    396272   1% /run
/dev/sda1      ext4      82083148 16935856  60931744  22% /
tmpfs          tmpfs      1987352        4   1987348   1% /dev/shm
none           tmpfs         1024        0      1024   0% /run/credentials/systemd
tmpfs          tmpfs      1987356       20   1987336   1% /tmp
none           tmpfs         1024        0      1024   0% /run/credentials/getty@t
tmpfs          tmpfs       397468      108    397360   1% /run/user/1000
```

5. Globbing

Create a dedicated practice directory containing sample files such as:

index.html, login.html, admin.htm, report.txt, access.log, error.log, user1.txt, user2.log, file123, backup1.html, backup2.txt

Use **globbing patterns wherever possible**, rather than specifying filenames individually.

a. List all files ending in .html.

```
ls *.html
```

b. List all files **other than .html files** using extended globbing.

```
ls !(*.html)
```

c. List filenames containing at least one numeric digit.

```
ls *[0-9]
```

d. List files whose names begin with user followed by a single digit.

```
ls user?.*
```

e. List files ending in either .txt or .log using a single **extended glob pattern**.

```
ls *.@(txt|log)
```

f. List files that do **not** end in .txt or .log.

```
ls !(*.txt|*.log)
```

g. List files beginning with either access or error using extended globbing.

```
ls @(access|error).log
```

h. List .html files whose filenames contain a numeric digit.

```
ls *[0-9]*.html
```

```
┌──(kali㉿kali)-[~]
└─$ bash
┌──(kali㉿kali)-[~]
└─$ shopt -s extglob

┌──(kali㉿kali)-[~]
└─$ mkdir globbing_practice

┌──(kali㉿kali)-[~]
└─$ cd globbing_practice

┌──(kali㉿kali)-[~/globbing_practice]
└─$ touch index.html login.html admin.htm report.txt access.log error.log user1.txt user2.log file123 backup1.html backu
p2.txt

┌──(kali㉿kali)-[~/globbing_practice]
└─$ ls
access.log  backup1.html  error.log  index.html  report.txt  user2.log
admin.htm   backup2.txt   file123    login.html  user1.txt

┌──(kali㉿kali)-[~/globbing_practice]
└─$ ls *.html
backup1.html  index.html  login.html

┌──(kali㉿kali)-[~/globbing_practice]
└─$ ls !(*.html)
access.log  admin.htm  backup2.txt  error.log  file123  report.txt  user1.txt  user2.log
```

```
┌──(kali㉿kali)-[~/globbing_practice]
└─$ ls *[0-9]*
backup1.html  backup2.txt  file123  user1.txt  user2.log

┌──(kali㉿kali)-[~/globbing_practice]
└─$ ls user?.*
user1.txt  user2.log

┌──(kali㉿kali)-[~/globbing_practice]
└─$ ls *.@(txt|log)
access.log  backup2.txt  error.log  report.txt  user1.txt  user2.log

┌──(kali㉿kali)-[~/globbing_practice]
└─$ ls !(*.txt|*.log)
admin.htm  backup1.html  file123  index.html  login.html

┌──(kali㉿kali)-[~/globbing_practice]
└─$ ls @(access|error).log
access.log  error.log

┌──(kali㉿kali)-[~/globbing_practice]
└─$ ls *[0-9].html
backup1.html
```

6. Extended Globbing

a. Assume a directory contains:

access.log, error.log, auth.log, auth.log.1, auth.log.2, system.log, notes.txt

Write an extended glob pattern that selects only access.log, error.log, and auth.log.

```
ls @(access|error|auth).log
```

Select all files except notes.txt.

```
ls !(notes.txt)
```

Select files named either auth.log.1 or auth.log.2 using pattern matching rather than writing both complete filenames separately.

```
ls auth.log.@(1|2)
```

b. Given the files:

malware1.log, malware2.log, malwareA.log, malware_report.txt, normal.log

construct a glob pattern that identifies .log files whose names start with malware and contain a numeric character.

```
ls malware*[0-9]*.log
```

c. Given:

failed_login.log, successful_login.log, login_backup.log, login.txt, firewall.log

use extended globbing to select .log files containing either failed_login or successful_login.

```
ls @(failed_login|successful_login).log
```

Construct a pattern that lists all .log files **except** firewall.log.

```
ls !("firewall.log")
```

```
┌──(kali㉿kali)-[~]
└─$ mkdir extended

┌──(kali㉿kali)-[~]
└─$ cd extended

┌──(kali㉿kali)-[~/extended]
└─$ touch access.log error.log auth.log auth.log.1 auth.log.2 system.log note
s.txt

┌──(kali㉿kali)-[~/extended]
└─$ ls
access.log  auth.log.1  error.log  system.log
auth.log    auth.log.2  notes.txt

┌──(kali㉿kali)-[~/extended]
└─$ ls @(access|error|auth).log
access.log  auth.log  error.log

┌──(kali㉿kali)-[~/extended]
└─$ ls !(notex.txt)
access.log  auth.log.1  error.log  system.log
auth.log    auth.log.2  notes.txt

┌──(kali㉿kali)-[~/extended]
└─$ ls auth.log.@(1|2)
auth.log.1  auth.log.2
```

```
┌──(kali㉿kali)-[~/extended]
└─$ touch malware1.log malware2.log malwareA.log malware_report.txt normal.lo
g

┌──(kali㉿kali)-[~/extended]
└─$ ls malware*[0-9]*.log
malware1.log  malware2.log

┌──(kali㉿kali)-[~/extended]
└─$ touch failed_login.log successful_login.log login_backup.log login.txt fi
rewall.log

┌──(kali㉿kali)-[~/extended]
└─$ ls @(failer_login|successful_login).log
successful_login.log

┌──(kali㉿kali)-[~/extended]
└─$ ls @(failed_login|successful_login).log
failed_login.log  successful_login.log

┌──(kali㉿kali)-[~/extended]
└─$ ls !("firewall.log")
access.log  error.log         malware1.log        normal.log
auth.log    failed_login.log  malware2.log        notes.txt
auth.log.1  login_backup.log  malwareA.log        successful_login.log
auth.log.2  login.txt         malware_report.txt  system.log
```

7. Filters

a. Display the first 10 entries of login.log.
head -10 login.log

b. Display the first 5 entries of access.log.
head -5 access.log

c. Display the first 15 security events from security.log.
head -15 security.log

d. Display the first 20 events recorded in incident.log.
head -20 incident.log

e. Display only the first entry in login.log.
head -1 login.log

f. Display the first 25 web requests recorded in access.log.
head -25 access.log

g. Display the last 10 entries of login.log.
tail -10 login.log

h. Display the last 5 entries of access.log.
tail -5 access.log

i. Display the 20 most recent security events from security.log.
tail -20 security.log

j. Display the 25 most recent events from incident.log.
tail -25 incident.log

k. Display only the last entry in security.log.
tail -1 security.log

l. Display the last 50 events recorded in incident.log.
tail -50 incident.log

m. Display the contents of login.log starting from line 41 until the end of the file.
tail -n +41 login.log

n. Display the contents of security.log starting from line 101 until the end of the file.
tail -n +101 security.log

o. Find the total number of lines in login.log.
wc -l login.log

p. Find the total number of lines in access.log.
wc -l access.log

q. Find the total number of security events in security.log.
wc -l security.log

r. Determine the total number of events recorded in incident.log.
wc -l incident.log

s. Count the number of words in login.log.
wc -w login.log

t. Count the number of bytes in security.log.
wc -c security.log

u. Display the line, word, and byte counts of access.log.
wc access.log

v. Find the line counts of all four .log files using a single wc command.
wc -l *.log

w. Display the first 20 entries of login.log and, from those entries, display only the last 5.

head -20 login.log | tail -5

x. Display the last 20 entries of login.log and, from those entries, display only the first 5.
tail -20 login.log | head -5

y. Display the first 30 entries of access.log and then display only the last 10 entries from that output.
head -30 access.log | tail -10

z. Display the last 30 entries of access.log and then display only the first 10 entries.
tail -30 access.log | head -10

aa. Display the first 50 events from security.log and then display the last 10 events from those 50.
head -50 security.log | tail -10

bb. Display the last 50 events from security.log and then display the first 10 events from those 50.
tail -50 security.log | head -10

cc. Display entries from line 11 to line 20 of login.log.
head -20 login.log | tail -10

dd. Display entries from line 21 to line 30 of login.log.
head -30 login.log | tail -10

ee. Display entries from line 11 to line 20 of access.log.
head -20 access.log | tail -10

ff. Display entries from line 31 to line 40 of access.log.
head -40 access.log | tail -10

gg. Display security events from line 21 to line 30 of security.log.
head -30 security.log | tail -10

hh. Display security events from line 51 to line 60 of security.log.
head -60 security.log | tail -10

ii. Display incident events from line 101 to line 110 of incident.log.
head -110 incident.log | tail -10

jj. Display incident events from line 201 to line 220 of incident.log.
head -220 incident.log | tail -20

kk. Display incident events from line 491 to line 500.
head -500 incident.log | tail -10

ll. Take the first 20 entries of login.log and count how many lines are displayed.
head -20 login.log | wc -l

mm. Take the last 15 entries of access.log and count how many lines are displayed.
tail -15 access.log | wc -l

nn. Take the first 50 entries of security.log and count the number of words in those entries.
head -50 security.log | wc -w

oo. Take the last 25 entries of security.log and count the number of lines.
tail -25 security.log | wc -l

pp. Extract lines 21–30 from login.log and count the number of extracted lines.
head -30 login.log | tail -10 | wc -l

qq. Extract lines 51–60 from access.log and count the number of extracted lines.
head -60 access.log | tail -10 | wc -l

rr. Extract events 101–150 from incident.log and count how many events are displayed.
head -150 incident.log | tail -50 | wc -l

ss. Display the first 100 events from incident.log, take the last 20 events from that output, and count the number of lines.

head -100 incident.log | tail -20 | wc -l

```
┌──(kali㉿kali)-[~]
└─$ mkdir filters

┌──(kali㉿kali)-[~]
└─$ cd filters

┌──(kali㉿kali)-[~/filters]
└─$ for i in {1..100}; do echo "Login Event $i"; done > login.log

┌──(kali㉿kali)-[~/filters]
└─$ for i in {1..120}; do echo "Access Request $i"; done > access.log

┌──(kali㉿kali)-[~/filters]
└─$ for i in {1..200}; do echo "Security Event $i"; done > security.log

┌──(kali㉿kali)-[~/filters]
└─$ for i in {1..500}; do echo "Incident Event $i"; done > incident.log

┌──(kali㉿kali)-[~/filters]
└─$ ls
access.log  incident.log  login.log  security.log

┌──(kali㉿kali)-[~/filters]
└─$ head -10 login.log
Login Event 1
Login Event 2
Login Event 3
Login Event 4
```